

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Кафедра экономической информатики

Дисциплина: Программирование сетевых приложений

**Отчёт о выполнении
лабораторной работы №2**
Вариант №4

Выполнял:
Биза Г.А.
гр. 414302

Проверил:
Иномов С. Ф.

Минск 2026

Цель работы: научиться создавать консольные приложения на Java, освоить принципы структурирования кода, обработки ввода/вывода, исключений и логирования с использованием библиотеки Log4j.

Задачи:

- изучить расширенные возможности классов в Java: поля, методы, конструкторы, модификаторы доступа;
- реализовать иерархию классов с использованием наследования;
- продемонстрировать переопределение методов и применение аннотации @Override;
- использовать полиморфизм для работы с объектами разных классов через общий интерфейс;
- реализовать консольное приложение, демонстрирующее работу наследования, полиморфизма и инкапсуляции;
- провести тестирование программы и сделать выводы о применении принципов ООП.

Теоретические сведения

Консольное приложение – программа, которая взаимодействует с пользователем через текстовый интерфейс: ввод с клавиатуры и вывод в консоль.

Java – строго типизированный язык, и каждая переменная имеет тип. Переменные – именованные области памяти. Основные типы переменных:

- int – целое число;
- double – число с плавающей запятой;
- boolean – логическое значение;
- char – символ;
- String – строка.

Массив – структура данных, хранящая фиксированное количество элементов одного типа.

Ввод и вывод данных – основа взаимодействия пользователя с программой. В Java для ввода используется класс Scanner, а для вывода – System.out.

Для чтения данных с клавиатуры используется класс Scanner.

Исключения – события, нарушающие нормальное выполнение программы. Обработка исключений позволяет избежать аварийного завершения и обеспечить устойчивость приложения. К ключевым конструкциям относятся блоки: try, catch и finally.

Логирование – процесс записи событий, ошибок и действий программы. Оно помогает отслеживать поведение приложения, выявлять ошибки и анализировать производительность.

Log4j – популярная библиотека для логирования в Java. Она позволяет записывать информацию о работе программы в консоль или файл, классифицируя сообщения по уровням важности. Уровни логирования:

- DEBUG – подробная отладочная информация;
- INFO – общая информация о ходе работы;
- WARN – предупреждения;
- ERROR – ошибки, не приводящие к завершению;
- FATAL – критические ошибки.

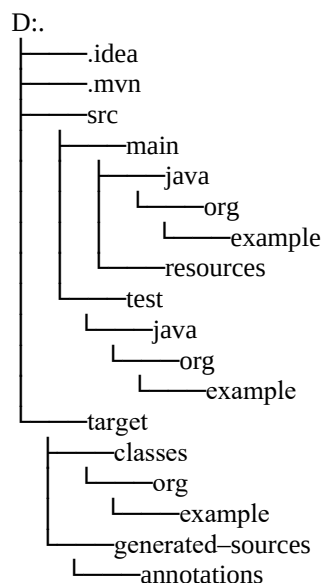
Методы – блоки кода, выполняющие определённую задачу. Они повышают читаемость и переиспользуемость.

Хорошо структурированный код облегчает сопровождение, тестирование и повторное использование. В консольных приложениях важно разделять:

- ввод/вывод;
- логика обработки;
- логирование;
- обработка ошибок.

Ход выполнения работы

Структура проекта организована по примеру из методического пособия и выглядит следующим образом:



Класс `Product` является абстрактной базовой моделью для всех товаров в системе магазина и отвечает за хранение общих атрибутов, присущих любому продукту: название (`name`), цена (`price`), категория (`category`) и количество единиц на складе (`quantity`). Реализуя интерфейс `Serializable`, он обеспечивает возможность сохранения состояния объектов в файл. Основные функции класса включают инициализацию этих данных через конструктор, предоставление доступа к ним с помощью геттеров и сеттеров, а также формирование базового строкового представления товара в переопределённом методе `toString()`, которое затем расширяется в классах-наследниках. Класс задаёт единый контракт для всех конкретных типов товаров

(Food, Clothes, Electronic) и инкапсулирует общую логику работы с их основными характеристиками.

```
package org.example;

import java.io.Serializable;

public abstract class Product implements Serializable {
    private String name;
    private int price;
    private final Category category;
    private int quantity;

    Product(String name, int price, Category category, int quantity){
        this.name = name;
        this.price = price;
        this.category = category;
        this.quantity = quantity;
    }

    public int getPrice() {
        return price;
    }

    public Category getCategory() {
        return category;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString(){
        return String.format("%S (категория: %S, цена: %d, количество: %d)",
            name,
            category.getDisplayName(),
            price,
            quantity
        );
    }
}
```

Классы «Food», «Clothes» и «Electronic» являются специализированными наследниками абстрактного класса «Product» и расширяют его функциональность для конкретных категорий товаров. Каждый из них добавляет уникальные атрибуты и методы, характерные для соответствующего типа продукции.

Класс «Food» моделирует продукты питания. Он добавляет поля «manufacturedDate» (дата производства) и «expiredDate» (срок годности), причём последний вычисляется автоматически на основе переданного количества дней годности. Класс предоставляет методы для проверки свежести: «isExpired()» определяет, просрочен ли товар; «daysUntilExpire()» возвращает количество дней до истечения срока; «getStatus()» классифицирует состояние товара (ПРОСРОЧЕН, КРИТИЧЕСКИЙ, ВНИМАНИЕ, НОРМАЛЬНЫЙ) в зависимости от оставшегося времени. Переопределённый метод «toString()» включает всю информацию о продукте, включая даты производства и годности (форматированные в «dd.MM.yyyy»), а также текущий статус, обеспечивая полное представление о товаре.

Класс «Clothes» представляет одежду. Он вводит поле «brand» (бренд) с соответствующими геттером и сеттером. Конструктор инициализирует общие поля через вызов родительского конструктора и устанавливает бренд. Переопределённый «toString()» дополняет базовое описание товара строкой с информацией о бренде, что позволяет однозначно идентифицировать производителя.

Класс «Electronic» описывает электронные приборы. Он добавляет поле «guaranteeTime» (срок гарантии в днях) с методами доступа. Конструктор принимает все общие параметры и гарантийный срок. Переопределённый «toString()» расширяет стандартный вывод информацией о гарантии, что важно для покупателей при выборе техники.

Все три класса инкапсулируют специфическую логику, характерную для своей категории, и обеспечивают расширенное представление товаров при выводе, наследуя при этом общие атрибуты и поведение от базового класса «Product».

```
package org.example;

import java.text.SimpleDateFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Food extends Product{
    private LocalDate manufacturedDate;
    private LocalDate expiredDate;

    public Food(String name, int price, Category category, int quantity,
        LocalDate manufacturedDate,
        int plusSomeDays){
        super(name, price, category, quantity);
        this.manufacturedDate = manufacturedDate;
        this.expiredDate = manufacturedDate.plusDays(plusSomeDays);
    }
}
```

```

    }

    public LocalDate getManufacturedDate() {
        return manufacturedDate;
    }

    public LocalDate getExpiredDate() {
        return expiredDate;
    }

    public boolean isExpired(){
        return LocalDate.now().isAfter(expiredDate);
    }

    public long daysUntilExpire(){
        return ChronoUnit.DAYS.between(LocalDate.now(),expiredDate);
    }

    public String getStatus(){
        long days = daysUntilExpire();
        if(days < 0){
            return "ПРОСРОЧЕН";
        } else if (days <= 3) {
            return "КРИТИЧЕСКИЙ (осталось " + days + " дн.)";
        } else if (days <= 7) {
            return "ВНИМАНИЕ (осталось " + days + " дн.)";
        }else {
            return "НОРМАЛЬНЫЙ";
        }
    }

    @Override
    public String toString() {
        DateTimeFormatter formatter =
            DateTimeFormatter.ofPattern("dd.MM.yyyy");
        return super.toString() + String.format(", произведен: %S, годен до: %S, статус: %S",
            manufacturedDate.format(formatter),
            expiredDate.format(formatter),
            getStatus());
    }
}

package org.example;

public class Clothes extends Product{

    private String brand;

    Clothes(String name, int price, Category category, int quantity, String brand){
        super(name, price, category, quantity);
        this.brand = brand;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override

```

```

        public String toString(){
            return super.toString() + String.format(
                " , бренд: %s", brand
            );
        }
    }
}
package org.example;

public class Electronic extends Product{

    private int guaranteeTime;

    Electronic(String name, int price, Category category, int quantity, int
guaranteeTime){
        super(name, price, category, quantity);
        this.guaranteeTime = guaranteeTime;
    }

    public int getGuaranteeTime() {
        return guaranteeTime;
    }

    public void setGuaranteeTime(int guaranteeTime) {
        this.guaranteeTime = guaranteeTime;
    }

    @Override
    public String toString(){
        return super.toString() + String.format(" , гарантия %d дн. ",
guaranteeTime);
    }
}
}

```

Класс «Shop» выступает в роли центрального репозитория и менеджера товаров в приложении. Он хранит статическую коллекцию «productList» (список объектов «Product»), которая содержит все товары, доступные в магазине, и предоставляет статические методы для управления этой коллекцией. Основные функции класса включают добавление («add»), удаление («remove»), изменение цены («changePrice»), а также различные операции поиска и фильтрации: поиск по названию («findProductsByName», «findFirstProductByName»), фильтрация по точной цене («filterByPrice») и диапазону цен («filterByRangePrice»). Для удобства навигации реализованы методы сортировки по названию («sortByProductName»), цене («sortByProductPrice») и категории («sortByProductCategory»). Метод «categoryFilter» выводит товары выбранной категории, а «showStatistics» подсчитывает и отображает общую информацию: количество товаров каждого типа, общее количество единиц и среднюю цену. Большинство операций реализовано с использованием Stream API, что обеспечивает функциональный и декларативный подход к обработке данных. Класс инкапсулирует логику хранения и обработки товаров, предоставляя единую точку доступа к ним для всего приложения.

```
package org.example;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.logging.LogManager;
import java.util.logging.Logger;
import java.util.stream.Collectors;

public class Shop {
    private static List<Product> productList = new ArrayList<>();

    public static void add(Product product){
        productList.add(product);
    }

    public static List<Product> getProductList(){
        return productList;
    }

    public static void remove(Product product){
        productList.remove(product);
    }

    public static void changePrice(Product product, int newPrice){
        product.setPrice(newPrice);
    }

    public static List<Product> findProductsByName(String productsName){
        return productList.stream()
            .filter(product ->
                product.getName().equalsIgnoreCase(productsName))
            .collect(Collectors.toList());
    }

    public static Optional<Product> findFirstProductByName(String
productName){
        return productList.stream()
            .filter(product ->
                product.getName().equalsIgnoreCase(productName))
            .findFirst();
    }

    public static List<Product> filterByPrice(int price){
        return productList.stream()
            .filter(product -> product.getPrice() == price)
            .collect(Collectors.toList());
    }

    public static List<Product> filterByRangePrice(int minValue, int
maxValue){
        return productList.stream()
            .filter(product -> product.getPrice() > minValue &&
                product.getPrice() < maxValue)
            .collect(Collectors.toList());
    }

    public static void showStatistics(){
        long clothesCount = productList.stream().filter(product -> product
instanceof Clothes).count();
        long foodCount = productList.stream().filter(product -> product
instanceof Food).count();
    }
}
```



```

        long electronicsCount = productList.stream().filter(product ->
product instanceof Electronic).count();

        long allItemQuantity =
productList.stream().mapToInt(Product::getQuantity).sum();

        double averagePriceInShop =
productList.stream().mapToInt(Product::getPrice).average().orElse(0);

        System.out.println("Одежды в наличии(предметов): " + clothesCount);
        System.out.println("Общее количество продуктов питания в наличии: " +
foodCount);
        System.out.println("Общее количество электронных приборов: " +
electronicsCount);
        System.out.println("Общее количество предметов в магазине: " +
allItemQuantity);
        System.out.println("Средняя стоимость товаров в магазине: " +
averagePriceInShop);
    }

    public static void sortByProductName(){
        productList.sort(Comparator.comparing(Product::getName,
String.CASE_INSENSITIVE_ORDER));
    }

    public static void sortByProductPrice(){
        productList.sort(Comparator.comparing(Product::getPrice));
    }

    public static void categoryFilter(int choice){
        switch (choice){
            case 1 ->{
                System.out.println("Вы выбрали категорию продукты питания:");
                List<Product> foodList = productList.stream().filter(product
-> product instanceof Food).toList();
                for(Product product : foodList){
                    System.out.println("1. " + product.getName() + "
стоимость: " + product.getPrice() +
" количество единиц товара: " +
product.getQuantity());
                }
            }
            case 2 ->{
                System.out.println("Вы выбрали категорию одежда:");
                List<Product> foodList = productList.stream().filter(product
-> product.getCategory().equals(Category.CLOTHES)).toList();
                for(Product product : foodList){
                    System.out.println("1. " + product.getName() + "
стоимость: " + product.getPrice() +
" количество единиц товара: " +
product.getQuantity());
                }
            }
            case 3 ->{
                System.out.println("Вы выбрали категорию электроника:");
                List<Product> electronicList =
productList.stream().filter(product ->
product.getCategory().equals(Category.ELECTRONICS)).toList();
                for(Product product : electronicList){
                    System.out.println("1. " + product.getName() + "
стоимость: " + product.getPrice() +
" количество единиц товара: " +
product.getQuantity());
                }
            }
        }
    }

```

```

    }

    };
}

public static void sortByProductCategory(){
    productList.sort(Comparator.comparing(Product::getCategory));
}
}

```

Выберете сценарий работы программы:

1. Добавить продукт питания
2. Добавить электронику
3. Добавить одежду
4. Просмотреть весь ассортимент
5. Удалить товар
6. Редактировать цену
7. Редактировать гарантию
8. Поиск по названию
9. Фильтр по категории
10. Фильтр по цене
11. Фильтр по диапазону цен
12. Сортировка по названию
13. Сортировка по цене
14. Статистика по категориям
15. Сохранение данных
16. Загрузка данных
0. Выход из программы

16

Выберете сценарий работы программы:

Рисунок 1 – результат работы консольного приложения

Вывод: в ходе выполнения лабораторной работы разработано консольное приложение на языке Java, моделирующее работу магазина с возможностью добавления, удаления, редактирования и просмотра товаров различных категорий (продукты питания, одежда, электроника). Реализованы функции поиска, фильтрации, сортировки, статистики, а также аутентификация для защищённых операций и сохранение/загрузка данных в файл с помощью сериализации. Приложение построено с соблюдением принципов объектно-ориентированного программирования, разделения

ответственности и обработки исключений, обеспечивая устойчивость к некорректному вводу. В процессе работы освоены: структурирование кода на классы, работа с коллекциями и Stream API, перечисления, наследование, полиморфизм, сериализация, организация пользовательского ввода с валидацией.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое консольное приложение в Java?

Это программа, которая работает в командной строке без графического интерфейса. Она принимает ввод с клавиатуры и выводит текст на экран.

2. Какова структура метода «main()» и его роль в приложении?

Метод «main()» – точка входа в программу: «public static void main(String[] args)». JVM вызывает его первой для запуска приложения.

3. Какие классы используются для ввода данных с клавиатуры?

Основные классы: «Scanner» (простой и удобный) и «BufferedReader» (эффективный для больших объёмов). Оба читают данные из «System.in».

4. Чем отличается «System.out.print()» от «System.out.println()»?

«print()» выводит текст без перехода на новую строку. «println()» после вывода автоматически переносит курсор на следующую строку.

5. Какие типы данных являются примитивными в Java?

В Java 8 примитивных типов: «byte», «short», «int», «long», «float», «double», «char», «boolean». Они хранят значение напрямую, а не ссылку на объект.

6. Как объявить и инициализировать массив в Java?

Можно объявить массив и выделить память: «int[] arr = new int[5]». Или сразу заполнить значениями: «int[] arr = {1, 2, 3}».

7. Как работает цикл «for–each» и когда он применяется?

Цикл автоматически перебирает все элементы массива или коллекции без использования индексов. Он удобен для чтения данных, когда не нужно изменять элементы.

8. Что такое условный оператор «if–else» и как он используется?

Оператор выполняет разные блоки кода в зависимости от условия. Если условие истинно – выполняется блок «if», иначе – «else».

9. Как обрабатываются исключения в Java? Приведите пример.

Исключения обрабатываются блоком «try–catch». В «try» помещается опасный код, в «catch» – его обработка, например: «try { int x = 10/0; } catch (ArithmeticException e) { System.out.println("Ошибка"); }».

10. Какие типы исключений бывают в Java?

Исключения делятся на проверяемые (checked, требуют обработки), непроверяемые (unchecked, ошибки логики) и ошибки (errors, критические сбои JVM).

11. Что такое класс и объект в объектно–ориентированном программировании?

Класс – это шаблон или чертёж, описывающий состояние и поведение. Объект – конкретный экземпляр класса, созданный по этому чертежу в памяти.

12. Как определить и вызвать метод в Java?

Метод определяется с указанием модификатора, возвращаемого типа и параметров, например: «int sum(int a, int b)». Вызывается он через объект или имя класса (для статических методов).

13. Что такое перегрузка методов и как она реализуется?

Перегрузка – это несколько методов с одинаковым именем, но разными параметрами. Компилятор сам выбирает нужный метод по типу и количеству аргументов.

14. Какие уровни логирования поддерживает Log4j?

Log4j поддерживает уровни от наиболее детального к критическому: TRACE, DEBUG, INFO, WARN, ERROR, FATAL. Это позволяет гибко настраивать детализацию логов.

15. Как подключить Log4j к Java-проекту?

Проще всего добавить зависимости через Maven или Gradle. Также можно скачать JAR-файлы и вручную подключить их к проекту.

16. Что содержит файл «log4j.properties» и как он настраивается?

Файл определяет, куда писать логи (аппендеры), в каком формате и для каких уровней. Например, можно настроить вывод в консоль и в файл с указанием шаблона сообщения.

17. Как создать логгер и использовать его в классе?

Нужно получить экземпляр логгера через «LogManager.getLogger(Класс.class)», а затем вызывать методы «logger.info()», «logger.error()» и т. д. для записи сообщений.

18. В чём преимущества использования логирования в приложениях?

Логирование помогает отслеживать ошибки без отладчика, анализировать поведение системы и вести аудит событий. Оно гибко настраивается и не снижает производительность, если отключено в продакшне.

19. Как организовать структуру проекта с несколькими классами?

Классы группируются по функциональности в пакеты (например, «model», «service», «repository»). Стандартная структура Maven/Gradle делит код на «main/java» (исходники) и «test/java» (тесты).